

JavaScript Interview Revision

Detailed explanations with examples • All core concepts • Service-based interview level

1. Execution Context & Call Stack

Execution Context

Definition:

An execution context is the environment in which JavaScript code is evaluated and executed. It's created every time a function is called or when the global code starts running. Think of it as a container that holds all the information needed to run code at that moment — variables, functions, the value of `this`, and references to the outer scope.

Two Phases:

1. Creation Phase: Variables and functions are hoisted; memory is allocated. Variables initialized with `undefined` (var) or left uninitialized (let/const). **2. Execution Phase:** Code runs line by line, values assigned, functions executed.

Example:

```
console.log(a); // undefined (creation phase allocated memory)
console.log(foo()); // 'hello' (function hoisted)

var a = 5;
function foo() { return 'hello'; }

console.log(a); // 5
```

Interview Tip: Understand that hoisting happens in the creation phase — that's why you get `undefined` instead of a `ReferenceError`.

Call Stack

Definition:

The call stack is a LIFO (Last-In-First-Out) data structure that tracks function execution. Every time a function is called, a new frame is pushed onto the stack. When the function returns, its frame is popped. The global execution context is always at the bottom.

Why It Matters:

Understanding the call stack explains recursion depth limits, stack overflow errors, and helps debug using browser DevTools. It also clarifies the order in which functions complete.

Example:

```
function a() { b(); }
function b() { c(); }
function c() { console.log('done'); }

a();
// Call Stack: [Global] → [a] → [b] → [c] → done → pop c → pop b → pop a
```

Interview Tip: Draw the call stack for tricky recursion problems. Show that each recursive call takes memory; too deep recursion → stack overflow.

Hoisting

Definition:

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before code execution. However, only declarations are hoisted, not initializations. `var`, `function`, and `class` are hoisted, but `let` and `const` are also hoisted but placed in the TDZ.

Important Distinctions:

var: Hoisted and initialized with `undefined`. **function:** Fully hoisted — can call before declaration. **let/const:** Hoisted but not initialized; in TDZ until the line of declaration.

Examples:

```
// var hoisting
console.log(x); // undefined (not an error!)
var x = 10;

// function hoisting
console.log(foo()); // 'hello' (works!)
function foo() { return 'hello'; }

// let/const (NOT accessible before declaration)
console.log(y); // ReferenceError: y is not defined
let y = 20;
```

Interview Tip: Be clear: hoisting doesn't mean code moves physically. The engine allocates memory for variables in the creation phase. The practical result looks like "hoisting," but it's really about memory allocation timing.

Temporal Dead Zone (TDZ)

Definition:

The TDZ is the time span between entering a scope and reaching the line where a `let` or `const` variable is declared. During this period, the variable exists but is not yet initialized, and accessing it throws a `ReferenceError`. This is by design to catch usage errors early.

Why It Matters:

TDZ prevents you from accidentally using a variable before it's declared, catching bugs that would silently return `undefined` with `var`. It's stricter and safer.

Example:

```
console.log(typeof x); // ReferenceError (in TDZ!)
let x = 5;

// But var:
console.log(typeof y); // undefined (safe)
var y = 5;

// TDZ in loops:
for(let i = 0; i < 3; i++) {
  console.log(i); // valid - i is initialized in this iteration
}
```

Interview Tip: Common interview trick: "What's the difference between `console.log(typeof x)` when `let x` is declared below?" Answer: `ReferenceError`, not `undefined`. This tests TDZ understanding.

2. Scope & Closures

Lexical Scope

Definition:

Lexical scope (also called static scope) means that the accessibility of variables is determined by the position of the functions and blocks in the code at write-time, not at run-time. The scope chain is fixed when the code is written.

Why It Matters:

JavaScript's lexical scoping is the foundation for closures and predictable variable resolution. It allows functions to "remember" the variables from their definition context, not their calling context. This is different from dynamic scope (used in some languages like Bash).

Example:

```
let global = 'global';

function outer() {
  let outer_var = 'outer';

  function inner() {
    let inner_var = 'inner';
    console.log(inner_var);    // 'inner'
    console.log(outer_var);    // 'outer' (lexical)
    console.log(global);      // 'global'
  }

  return inner;
}

const fn = outer();
fn(); // Runs in its lexically-defined scope
```

Interview Tip: Lexical scope is decided at parse-time (when code is written), not runtime (when code runs). This is why closures work.

Scope Chain

Definition:

The scope chain is the mechanism JavaScript uses to resolve variable names. When a variable is referenced, the engine looks it up starting in the current scope, then moving outward through each parent scope (up the lexical nesting), until it finds the variable or reaches the global scope. If not found anywhere, a `ReferenceError` is thrown.

Why It Matters:

Understanding the scope chain is critical for debugging variable reference errors, understanding closures, and recognizing when variables are shadowed (a name used in an inner scope overrides one in an outer scope).

Example:

```
let a = 'global';

function f1() {
  let a = 'f1';

  function f2() {
    let a = 'f2';
    console.log(a); // 'f2' (found in current scope)
  }

  f2();
  console.log(a); // 'f1' (f2's 'a' doesn't affect here)
}

f1();
console.log(a); // 'global'
```

Interview Tip: Variable shadowing: inner scopes can redefine outer variables. The inner one takes precedence. This can be confusing but is intentional.

Closures

Definition:

A closure is a function that "remembers" variables from its outer scope even after the outer function has returned. The function maintains references to its lexically-scoped variables through a closure object. Every function in JavaScript creates a closure.

Why It Matters:

Closures enable data privacy (encapsulation), are used for callbacks/event handlers, power memoization, create higher-order functions, and are essential in modern JavaScript patterns like Redux, React hooks, and currying. Understanding closures is critical for async code and complex state management.

Classic Example — Counter:

```
function makeCounter() {
  let count = 0; // private variable

  return {
    increment() { return ++count; },
    decrement() { return --count; },
    get() { return count; }
  };
}

const counter = makeCounter();
console.log(counter.increment()); // 1
console.log(counter.increment()); // 2
console.log(counter.get());       // 2
// count is private – no way to access it directly
```

Memoization Example:

```
function memoize(fn) {
  const cache = {}; // private cache (closure)

  return function(...args) {
    const key = JSON.stringify(args);
    if (key in cache) {
      console.log('cached');
      return cache[key];
    }
    const result = fn(...args);
    cache[key] = result;
    return result;
  };
}

const add = memoize((a, b) => a + b);
console.log(add(2, 3)); // 5 (computed)
console.log(add(2, 3)); // 5 (cached)
```

Interview Tip: Classic question: "What is a closure?" Answer: A closure is a function bundled with its outer scope variables. Every function in JS is a closure. Show examples of data privacy and memoization.

3. The this Keyword & Function Binding

The this Keyword

Definition:

The value of `this` is determined by how a function is called, not where it's defined. `this` refers to the object that is executing the current function. In different calling contexts, `this` can refer to different objects — this flexibility is powerful but also confusing.

Rules for this binding (in order):

1. **new keyword:** `this` = newly created object
2. **call/apply/bind:** `this` = explicitly passed object
3. **Method call (obj.method()):** `this` = obj
4. **Arrow function:** `this` = lexically inherited from enclosing scope
5. **Regular function call:** `this` = undefined (strict) or global (non-strict)

Example — Method Call:

```
const person = {
  name: 'Alice',
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};

person.greet(); // 'Hi, I'm Alice' (this = person)

const greet = person.greet;
greet(); // TypeError or 'undefined' (this = global/undefined)
```

Interview Tip: The most common gotcha: passing a method as a callback loses its `this` context. Always use arrow functions or `.bind()` in callbacks.

call(), apply(), bind()

Definition:

These three methods allow you to explicitly set the value of `this` when a function runs. They are used to "borrow" methods from other objects or lock in a context for later execution.

call(thisArg, arg1, arg2, ...): Invokes the function immediately, passing arguments as a list.

apply(thisArg, [arg1, arg2, ...]): Same as call, but arguments are passed as an array.

bind(thisArg, arg1, arg2, ...): Returns a new function with `this` fixed; doesn't invoke immediately. Supports partial application.

Example — call & apply:

```
function greet(greeting, suffix) {
  console.log(`${greeting}, I'm ${this.name}${suffix}`);
}

const person = { name: 'Bob' };

// call — args as list
greet.call(person, 'Hello', '!'); // 'Hello, I'm Bob!'

// apply — args as array
greet.apply(person, ['Hi', '!!!']); // 'Hi, I'm Bob!!!'

// Real-world: borrowing Array methods
const arr = [1, 2, 3];
const max = Math.max.apply(null, arr); // 3
```

Example — bind:

```
function getName() {
  return this.name;
}

const person1 = { name: 'Alice' };
const person2 = { name: 'Bob' };

const getAliceName = getName.bind(person1);
console.log(getAliceName()); // 'Alice' (always)

const addFive = (a,b,c) => a+b+c;
const addFiveAndTwo = addFive.bind(null, 5);
console.log(addFiveAndTwo(2, 3)); // 10
```

Interview Tip: Difference: `call/apply` invoke immediately; `bind` returns a bound function. Use `bind` in React class components or event listeners.

4. Event Loop & Async Execution

Event Loop

Definition:

The Event Loop is the core mechanism that allows JavaScript to be non-blocking despite running on a single thread. It continuously checks if the call stack is empty; if so, it moves tasks from the task queues (microtask queue first, then macrotask queue) to the call stack for execution. This cycle repeats infinitely.

Why It Matters:

Understanding the event loop is essential for debugging asynchronous code, optimizing performance, and understanding why callbacks execute in a specific order. It's almost always asked in interviews for async-heavy roles.

Order of Execution:

1. Call all synchronous code (global scope)
2. Check microtask queue (Promises, `async/await`)
3. Process ONE macrotask (`setTimeout`, events)
4. Go back to step 2

Interview Tip: Common question: "What's printed and in what order?" Always map out: call stack → microtask queue → macrotask queue. Draw it out in interviews.

Microtask Queue vs Macrotask Queue

Microtasks (High Priority):

Promise callbacks (`.then` , `.catch` , `.finally`), `async/await` , `queueMicrotask` , `MutationObserver`. The entire microtask queue is processed before moving to the next macrotask.

Macrotasks (Low Priority):

`setTimeout` , `setInterval` , `setImmediate` (Node.js), I/O operations, UI events (click, scroll), rendering. Only one macrotask is executed per event loop iteration; then all microtasks are run.

Example:

```
console.log('start');

setTimeout(() => console.log('timeout 1'), 0);

Promise.resolve()
  .then(() => console.log('promise 1'))
  .then(() => console.log('promise 2'));

setTimeout(() => console.log('timeout 2'), 0);

console.log('end');
```

// Output:
// start
// end
// promise 1
// promise 2
// timeout 1
// timeout 2

Interview Tip: Execution order: Synchronous → Microtasks (all) → One Macrotask → Microtasks (again) → Next Macrotask. This is crucial for Promise/`setTimeout` interactions.

Promises

Definition:

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its result. It has three states: **pending** (initial), **fulfilled** (resolved), **rejected** (errored). Once settled (fulfilled or rejected), a Promise cannot change states.

Why It Matters:

Promises solve callback hell (deeply nested callbacks), provide cleaner error handling with `.catch`, and are the foundation for modern async patterns like `async/await`. They make async code more readable and composable.

Example:

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    if (Math.random() > 0.5) {
      resolve('Success!');
    } else {
      reject('Failed!');
    }
  }, 1000);
});

promise
  .then(result => console.log(result))
  .catch(error => console.error(error))
  .finally(() => console.log('Done'));
```

Interview Tip: A `.then` that returns a new Promise will automatically "unwrap" it and chain. Understanding Promise chaining vs nesting is key.

Promise.all

Definition:

`Promise.all` takes an iterable of promises and returns a new promise. It resolves when ALL input promises have resolved, returning an array of results in the same order. If ANY promise rejects, the whole `Promise.all` rejects immediately (fail-fast).

Use Case:

Parallel execution of independent async operations where you need all results and any failure means the whole operation fails. Examples: fetching multiple resources, running multiple validations.

Example:

```
const p1 = fetch('/data/1').then(r => r.json());
const p2 = fetch('/data/2').then(r => r.json());
const p3 = fetch('/data/3').then(r => r.json());

Promise.all([p1, p2, p3])
  .then(([data1, data2, data3]) => {
    console.log('All data:', data1, data2, data3);
  })
  .catch(err => console.error('One failed:', err));
```

Interview Tip: Key difference from `Promise.allSettled`: `Promise.all` rejects immediately on first failure; `allSettled` waits for all.

Promise.race

Definition:

`Promise.race` takes multiple promises and settles with the result of whichever promise settles first (either resolves or rejects). The others are ignored after one settles.

Use Case:

Timeouts, competitive requests (fastest server), or first-to-complete scenarios. Common: racing a fetch against a timeout promise.

Example — Timeout Pattern:

```
function timeout(ms) {
  return new Promise( (_, reject) =>
    setTimeout(() => reject(new Error('Timeout')), ms)
  );
}

Promise.race([
  fetch('/data'),
  timeout(5000)
])
  .then(res => res.json())
  .catch(err => console.error('Request timed out'));
```

Interview Tip: Race is rarely used in practice. The timeout pattern is useful for real code.

Promise.allSettled

Definition:

`Promise.allSettled` waits for ALL promises to settle (resolve or reject). Returns an array of objects: `{ status: 'fulfilled', value: ... }` or `{ status: 'rejected', reason: ... }`. Never rejects.

Use Case:

When you need results from all operations regardless of failures. Example: batch API calls where you want to process successful ones and log failures separately.

Example:

```
const urls = ['/api/1', '/api/2', '/api/3'];
const requests = urls.map(url => fetch(url).then(r => r.json()));

Promise.allSettled(requests)
  .then(results => {
    results.forEach((result, index) => {
      if (result.status === 'fulfilled') {
        console.log(`[${index}] Success:`, result.value);
      } else {
        console.log(`[${index}] Failed:`, result.reason);
      }
    });
  });
```

Interview Tip: Use `allSettled` for batch operations where failures in some items shouldn't abort the whole process.

Async / Await

Definition:

`async` function automatically returns a Promise. `await` pauses execution until a Promise settles, and extracts its value (or throws if rejected). It's syntactic sugar over Promise chains, making async code look synchronous.

Why It Matters:

Async/await is cleaner than Promise chains, easier to read and debug, and makes error handling simple with try/catch. Nearly all modern async code uses async/await.

Basic Example:

```
async function fetchUser() {
  try {
    const res = await fetch('/api/user');
    const user = await res.json();
    console.log(user);
    return user;
  } catch (err) {
    console.error('Error:', err);
  }
}

fetchUser(); // Returns a Promise
```

Sequential vs Parallel:

```
// SLOW - Sequential (1s + 1s + 1s = 3s)
async function slowLoad() {
  const data1 = await fetch('/data/1').then(r => r.json());
  const data2 = await fetch('/data/2').then(r => r.json());
  const data3 = await fetch('/data/3').then(r => r.json());
}

// FAST - Parallel (all concurrent = 1s)
async function fastLoad() {
  const [data1, data2, data3] = await Promise.all([
    fetch('/data/1').then(r => r.json()),
    fetch('/data/2').then(r => r.json()),
    fetch('/data/3').then(r => r.json())
  ]);
}
```

Interview Tip: Common performance mistake: using `await` in loops instead of `Promise.all`. Know when to parallelize vs serialize.

5. Array Methods

map()

Definition:

`map(callback)` transforms each element using the callback function and returns a new array of the same length. The original array is never mutated. The callback receives `(element, index, array)`.

Example:

```
const nums = [1, 2, 3, 4];

// Simple transformation
const doubled = nums.map(n => n * 2); // [2, 4, 6, 8]

// Transform objects
const users = [{id:1, name:'Alice'}, {id:2, name:'Bob'}];
const names = users.map(u => u.name); // ['Alice', 'Bob']
```

Interview Tip: map creates a new array; original is unchanged. Essential for React rendering lists.

filter()

Definition:

`filter(callback)` returns a new array containing only elements where the callback returns a truthy value. It doesn't mutate the original.

Example:

```
const nums = [1, 2, 3, 4, 5];

// Filter evens
const evens = nums.filter(n => n % 2 === 0); // [2, 4]

// Chaining map + filter
const result = nums
  .filter(n => n > 2)
  .map(n => n * 2);
// [6, 8, 10]
```

Interview Tip: filter + map is a powerful pattern. In SQL: filter is WHERE and map is SELECT.

reduce()

Definition:

`reduce(callback, initialValue)` iterates through the array, accumulating a single value. The callback receives `(accumulator, currentValue, index, array)`.

Sum Example:

```
const nums = [1, 2, 3, 4];

// Sum all
const sum = nums.reduce((acc, n) => acc + n, 0); // 10
```

Grouping Example:

```
const items = [{type:'fruit', name:'apple'}, {type:'veg', name:'carrot'}];

// Group by type
const grouped = items.reduce((acc, item) => {
  const key = item.type;
  if (!acc[key]) acc[key] = [];
  acc[key].push(item.name);
  return acc;
}, {});
// {fruit: ['apple'], veg: ['carrot']}
```

Interview Tip: Reduce is the most versatile array method. Master it for sums, grouping, flattening, and building objects.

find() & findIndex()

find():

Returns the *first* element matching a condition, or `undefined`. Stops searching as soon as a match is found.

findIndex():

Returns the index of the first matching element, or `-1` if none found. Use when you need the position for removal/update.

Example:

```
const users = [{id:1, name:'Alice'}, {id:2, name:'Bob'}];

// Find
const user = users.find(u => u.id > 1); // {id:2, name:'Bob'}

// FindIndex
const idx = users.findIndex(u => u.id === 2); // 1
```

Interview Tip: `find` returns element; `findIndex` returns position. Choose based on what you need.

some() & every()

some():

Returns true if at least one element passes the test; false otherwise. Stops early on first true.

every():

Returns true only if all elements pass; stops early on first false.

Example:

```
const nums = [1, 2, 3, 4, 5];

// At least one even?
const hasEven = nums.some(n => n % 2 === 0); // true

// All positive?
const allPositive = nums.every(n => n > 0); // true
```

Interview Tip: These are boolean checks. More efficient than filter for validation. They short-circuit for performance.

sort()

Definition:

`sort(compareFn)` sorts the array in-place (mutates the original). Without a comparator, it converts elements to strings and sorts by Unicode code points.

Example — The Gotcha:

```
const nums = [10, 2, 1, 30];

// Wrong – string sort
console.log(nums.sort()); // [1, 10, 2, 30]

// Correct – numeric sort
console.log(nums.sort((a, b) => a - b)); // [1, 2, 10, 30]
```

Interview Tip: A classic gotcha: `[3,10,1].sort()` → `[1, 10, 3]` (string sort). Always provide a comparator for numbers.

slice() vs splice()

slice():

Returns a shallow copy of a portion (non-mutating). Start inclusive, end exclusive.

splice():

Removes/inserts elements in place (mutating). Returns the removed elements.

Example:

```
const arr = [1, 2, 3, 4, 5];

// slice - non-mutating
const sub = arr.slice(1, 3); // [2, 3], arr unchanged

// splice - mutating
const removed = arr.splice(1, 2); // [2, 3], arr now [1, 4, 5]
```

Interview Tip: In React/functional code, always use slice for immutability. splice mutates, which can cause bugs.

includes()

Definition:

`includes(searchElement)` returns true if the array contains the element, false otherwise. Uses SameValueZero comparison (handles NaN correctly, unlike `===`).

Example:

```
const arr = [1, 2, NaN, 3];

// includes handles NaN correctly
console.log(arr.includes(NaN)); // true

// indexOf doesn't handle NaN
console.log(arr.indexOf(NaN)); // -1 (wrong!)
```

Interview Tip: NaN gotcha: `includes(NaN)` true, but `indexOf(NaN)` returns -1. Know the difference.

6. Deep Copy & Shallow Copy

Shallow Copy

Definition:

A shallow copy duplicates only the top-level structure. Nested objects and arrays are not cloned — they are still referenced. Changes to nested data affect both the original and the copy.

Methods:

```
arr.slice(), [...arr], Object.assign({}, obj), {...obj}
```

Example:

```
const original = {name:'Alice', address:{city:'NY'}};

// Shallow copy using spread
const copy = {...original};

// Top-level changes don't affect original
copy.name = 'Bob'; // original.name still 'Alice' ✓

// But nested objects are still shared!
copy.address.city = 'LA'; // original.address.city also 'LA' ✗
```

Interview Tip: Shallow copy is the default. This is a common source of bugs in React state with nested objects.

Deep Copy

Definition:

A deep copy recursively clones all levels. No shared references between original and copy.

Methods:

`structuredClone(obj)` (modern, recommended), `JSON.parse(JSON.stringify(obj))` (simple but loses functions/undefined), Lodash `_cloneDeep(obj)`

Example:

```
const original = {name:'Alice', address:{city:'NY'}};

// Deep copy using structuredClone
const deepCopy = structuredClone(original);

deepCopy.address.city = 'LA';
console.log(original.address.city); // 'NY' ✓
```

Interview Tip: Use `structuredClone` in modern browsers. For older browsers or when preserving functions, use recursive cloning or a library.

7. ES6+ Features

Destructuring

Definition:

Destructuring unpacks values from arrays or objects into separate variables in a single statement. It's syntactic sugar that makes code cleaner and more readable.

Array Destructuring:

```
const arr = [1, 2, 3, 4];

// Basic
const [first, second] = arr; // first=1, second=2

// Skip elements
const [a, , c] = arr; // a=1, c=3

// Rest operator (collect remaining)
const [x, ...rest] = arr; // x=1, rest=[2,3,4]
```

Object Destructuring:

```
const user = {name:'Alice', age:25, city:'NY'};

// Basic
const {name, age} = user;

// Rename
const {name: fullName} = user;

// Nested
const {address:{city}} = {name:'Bob', address:{city:'LA'}};
```

Interview Tip: Destructuring is widely used in React (props, state), function parameters, and imports. Master it.

Spread Operator (...)

Definition:

The spread operator expands an iterable (array, object) into individual elements. Used for copying, merging, and passing arguments.

Array Spread:

```
const arr1 = [1, 2, 3];

// Copy
const copy = [...arr1]; // [1, 2, 3]

// Merge
const merged = [0, ...arr1, 4]; // [0, 1, 2, 3, 4]
```

Object Spread:

```
const obj1 = {a:1, b:2};

// Copy
const copy = {...obj1};

// Merge (right overwrites left)
const merged = {...obj1, b:3, c:4}; // {a:1, b:3, c:4}
```

Interview Tip: Spread is powerful for immutability in React. Use it instead of mutating arrays/objects.

Rest Operator (...)

Definition:

The rest operator collects multiple elements into a single array. It's the opposite of spread — whereas spread expands, rest collects.

Function Parameters:

```
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3, 4); // 10

// Mix regular params with rest
function greet(greeting, ...names) {
  console.log(greeting + ' ' + names.join(', '));
}
greet('Hi', 'Alice', 'Bob'); // 'Hi Alice, Bob'
```

Destructuring with Rest:

```
const [first, ...rest] = [1, 2, 3, 4];
// first=1, rest=[2, 3, 4]

const {a, ...other} = {a:1, b:2, c:3};
// a=1, other={b:2, c:3}
```

Interview Tip: Rest must be the last element. You can't have rest in the middle of destructuring.